МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. Шухова»
(БГТУ им. В. Г. Шухова)

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Лабораторная работа № 3
по дисциплине: «Исследование операций»
на тему «Модификации симплекс метода. Методы искусственного базиса и больших штрафов»

Выполнил: ст. группы ПВ-242
Кладченко Никита Сергеевич

Проверил:
Вирченко Юрий Петрович

Белгород, 2026

Цель работы

Изучение методов искусственного базиса и больших штрафов решения задач ЛП в канонической форме, не подготовленных к работе симплекс-методом в чистом виде.

Задание

1. Изучить метод и алгоритм искусственного базиса и составить программу решения задачи ЛП этим методом.
2. Изучить метод и алгоритм больших штрафов и составить программу решения задачи ЛП этим методом.
3. Запрограммировать изученные алгоритмы и отладить соответствующие программы. В рамках подготовки тестовых данных решить вручную одну из следующих ниже задач.

Вариант 8

8.

$$z = x_1 - 3x_2 + 4x_3 + 5x_4 - x_5 + 8x_6 \to \max;$$

$$\begin{cases} x_1 + 5x_2 - 3x_3 - 4x_4 + 2x_5 + x_6 = 14, \\ 2x_1 + 9x_2 - 5x_3 - 7x_4 + 4x_5 + 2x_6 = 32, \end{cases}$$

$$x_i \geq 0 \left(i = \overline{1, 6}\right).$$

Ход работы

Задание 1. Изучить метод и алгоритм искусственного базиса и составить программу решения задачи ЛП этим методом

Дана задача линейного программирования:

$$z = x_1 - 3x_2 + 4x_3 + 5x_4 - x_5 + 8x_6 \to \max$$

при ограничениях

$$\begin{cases} x_1 + 5x_2 - 3x_3 - 4x_4 + 2x_5 + x_6 = 14, \\ 2x_1 + 9x_2 - 5x_3 - 7x_4 + 4x_5 + 2x_6 = 32, \\ \qquad\qquad\qquad\qquad\qquad\qquad x_i \geq 0 \quad (i = 1, \ldots, 6). \end{cases}$$

Метод искусственного базиса (двухфазный симплекс-метод)

Вводим искусственные переменные $y_1 \geq 0,\ y_2 \geq 0$:

$$\begin{cases} x_1 + 5x_2 - 3x_3 - 4x_4 + 2x_5 + x_6 + y_1 = 14, \\ 2x_1 + 9x_2 - 5x_3 - 7x_4 + 4x_5 + 2x_6 + y_2 = 32. \end{cases}$$

Фаза 1. Минимизируем

$$w = y_1 + y_2,$$

или, что эквивалентно, максимизируем

$$z_1 = -w.$$

Исходная симплекс-таблица.

Базис	x_1	x_2	x_3	x_4	x_5	x_6	y_1	y_2	b
y_1	1	5	-3	-4	2	1	1	0	14
y_2	2	9	-5	-7	4	2	0	1	32
z_1	-3	-14	8	11	-6	-3	0	0	-46

Итерация 1. Самый отрицательный коэффициент в строке z_1 — у переменной x_2 (-14). Следовательно, ведущий столбец — x_2.

Тестовые отношения:

$$\frac{14}{5} = 2.8, \qquad \frac{32}{9} \approx 3.556.$$

Минимальное отношение равно 2.8, значит из базиса выходит y_1. Поворотный элемент равен 5.

Новая строка x_2 (бывшая строка y_1, делённая на 5):

$$\left(\frac{1}{5},\ 1,\ -\frac{3}{5},\ -\frac{4}{5},\ \frac{2}{5},\ \frac{1}{5},\ \frac{1}{5},\ 0 \,\middle|\, \frac{14}{5}\right)$$

Новая строка $y_2 =$ старая строка $y_2 - 9$ новая строка x_2:

$$\left(\frac{1}{5},\ 0,\ \frac{2}{5},\ \frac{1}{5},\ \frac{2}{5},\ \frac{1}{5},\ -\frac{9}{5},\ 1 \,\middle|\, \frac{34}{5}\right)$$

Новая строка $z_1 =$ старая строка $z_1 + 14$ новая строка x_2:

$$\left(-\frac{1}{5},\ 0,\ -\frac{2}{5},\ -\frac{1}{5},\ -\frac{2}{5},\ -\frac{1}{5},\ \frac{14}{5},\ 0 \,\middle|\, -\frac{34}{5}\right)$$

Таблица после итерации 1.

Базис	x_1	x_2	x_3	x_4	x_5	x_6	y_1	y_2	b
x_2	$\frac{1}{5}$	1	$-\frac{3}{5}$	$-\frac{4}{5}$	$\frac{2}{5}$	$\frac{1}{5}$	$\frac{1}{5}$	0	$\frac{14}{5}$
y_2	$\frac{1}{5}$	0	$\frac{2}{5}$	$\frac{1}{5}$	$\frac{2}{5}$	$\frac{1}{5}$	$-\frac{9}{5}$	1	$\frac{34}{5}$
z_1	$-\frac{1}{5}$	0	$-\frac{2}{5}$	$-\frac{1}{5}$	$-\frac{2}{5}$	$-\frac{1}{5}$	$\frac{14}{5}$	0	$-\frac{34}{5}$

Итерация 2. Самый отрицательный коэффициент в строке z_1 — у переменных x_3 и x_5, оба равны $-\frac{2}{5}$. Выберем ведущий столбец x_3.

Тестовые отношения проводим только по положительным элементам в столбце x_3:

- для строки x_2: коэффициент $-\frac{3}{5} < 0$, не учитывается;
- для строки y_2:

$$\frac{34/5}{2/5} = 17.$$

Следовательно, из базиса выходит y_2. Поворотный элемент равен $\frac{2}{5}$.

Новая строка x_3:

$$\left(\frac{1}{2},\ 0,\ 1,\ \frac{1}{2},\ 1,\ \frac{1}{2},\ -\frac{9}{2},\ \frac{5}{2}\ \middle|\ 17\right)$$

Новая строка x_2 = старая строка $x_2 + \frac{3}{5}$ новая строка x_3:

$$\left(\frac{1}{2},\ 1,\ 0,\ -\frac{1}{2},\ 1,\ \frac{1}{2},\ -\frac{5}{2},\ \frac{3}{2}\ \middle|\ 13\right)$$

Новая строка z_1 = старая строка $z_1 + \frac{2}{5}$ новая строка x_3:

$$(0,\ 0,\ 0,\ 0,\ 0,\ 0,\ 1,\ 1 \mid 0)$$

После этой итерации $z_1 = 0$, искусственные переменные выведены из базиса. Фаза 1 завершена.

Таблица после фазы 1.

Базис	x_1	x_2	x_3	x_4	x_5	x_6	b
x_2	$\frac{1}{2}$	1	0	$-\frac{1}{2}$	1	$\frac{1}{2}$	13
x_3	$\frac{1}{2}$	0	1	$\frac{1}{2}$	1	$\frac{1}{2}$	17

Фаза 2. Переходим к исходной целевой функции

$$z = x_1 - 3x_2 + 4x_3 + 5x_4 - x_5 + 8x_6.$$

После пересчёта строки цели получаем таблицу:

Базис	x_1	x_2	x_3	x_4	x_5	x_6	b
x_2	$\frac{1}{2}$	1	0	$-\frac{1}{2}$	1	$\frac{1}{2}$	13
x_3	$\frac{1}{2}$	0	1	$\frac{1}{2}$	1	$\frac{1}{2}$	17
z	$\frac{1}{2}$	0	0	$\frac{3}{2}$	-2	$\frac{15}{2}$	29

Итерация 3. Самый положительный коэффициент в строке z — у переменной x_6 $\left(\frac{15}{2}\right)$. Следовательно, ведущий столбец — x_6.

Тестовые отношения:

$$\frac{13}{1/2} = 26, \qquad \frac{17}{1/2} = 34.$$

Минимальное отношение равно 26, значит из базиса выходит x_2. Поворотный элемент равен $\frac{1}{2}$.

Новая строка x_6:

$$(1,\ 2,\ 0,\ -1,\ 2,\ 1 \mid 26)$$

Новая строка x_3:

$$(0,\ -1,\ 1,\ 1,\ 0,\ 0 \mid 4)$$

Новая строка z:

$$(-7,\ -15,\ 0,\ 9,\ -17,\ 0 \mid 224)$$

Таблица после итерации 3.

Базис	x_1	x_2	x_3	x_4	x_5	x_6	b
x_6	1	2	0	-1	2	1	26
x_3	0	-1	1	1	0	0	4
z	-7	-15	0	9	-17	0	224

Итерация 4. Самый положительный коэффициент в строке z — у переменной x_4 (9). Следовательно, ведущий столбец — x_4.

Тестовые отношения проводим только по положительным элементам столбца x_4:

- для строки x_6: коэффициент $-1 < 0$, не учитывается;
- для строки x_3:

$$\frac{4}{1} = 4.$$

Из базиса выходит x_3. Поворотный элемент равен 1.

Новая строка x_4:

$$(0,\ -1,\ 1,\ 1,\ 0,\ 0 \mid 4)$$

Новая строка x_6:

$$(1,\ 1,\ 1,\ 0,\ 2,\ 1 \mid 30)$$

Новая строка z:

$$(-7,\ -6,\ -9,\ 0,\ -17,\ 0 \mid 260)$$

Оптимальная таблица.

Базис	x_1	x_2	x_3	x_4	x_5	x_6	b
x_6	1	1	1	0	2	1	30
x_4	0	-1	1	1	0	0	4
z	-7	-6	-9	0	-17	0	260

Все коэффициенты в строке целевой функции ≤ 0, следовательно, решение оптимально.

Оптимальное решение.

$$x_4 = 4, \qquad x_6 = 30, \qquad x_1 = x_2 = x_3 = x_5 = 0,$$

$$z^* = 260.$$

Задание 2. Изучить метод и алгоритм больших штрафов и составить программу решения задачи ЛП этим методом

Вводим те же искусственные переменные $y_1,\ y_2$ и большую константу $M \gg 0$. Тогда целевая функция принимает вид:

$$z' = x_1 - 3x_2 + 4x_3 + 5x_4 - x_5 + 8x_6 - M(y_1 + y_2) \to \max\,.$$

Система ограничений:

$$\begin{cases} x_1 + 5x_2 - 3x_3 - 4x_4 + 2x_5 + x_6 + y_1 = 14, \\ 2x_1 + 9x_2 - 5x_3 - 7x_4 + 4x_5 + 2x_6 + y_2 = 32. \end{cases}$$

Первые итерации метода больших штрафов совпадают с вычислениями Фазы 1 метода искусственного базиса: искусственные переменные выводятся из базиса, после чего дальнейшие шаги полностью соответствуют обычному симплекс-методу для исходной задачи.

Следовательно, метод больших штрафов приводит к тому же оптимальному решению:

$$x_4 = 4, \qquad x_6 = 30, \qquad x_1 = x_2 = x_3 = x_5 = 0,$$

$$z^* = 260.$$

Задание 3. Запрограммировать изученные алгоритмы и отладить соответствующие программы

Код программы метода искусственного базиса:

```
import numpy as np

EPS = 1e-10


def pivot_tableau(tableau, pivot_row_index, pivot_col_index):
    pivot_value = tableau[pivot_row_index, pivot_col_index]
    tableau[pivot_row_index, :] = tableau[pivot_row_index, :] /
   pivot_value

    row_count = tableau.shape[0]
    for row_index in range(row_count):
        if row_index != pivot_row_index:
            factor = tableau[row_index, pivot_col_index]
            if abs(factor) > EPS:
                tableau[row_index, :] = tableau[row_index, :] -
   factor * tableau[pivot_row_index, :]


def choose_entering_variable_max(tableau):
    objective_row = tableau[-1, :-1]
    entering_col_index = int(np.argmin(objective_row))
    min_value = float(objective_row[entering_col_index])
```

```
    is_optimal = True
    if min_value < -1e-12:
        is_optimal = False

    return {
        "is_optimal": is_optimal,
        "entering_col_index": entering_col_index
    }


def choose_leaving_variable(tableau, entering_col_index):
    constraint_rows = tableau[:-1, :]
    rhs_column = constraint_rows[:, -1]
    entering_column = constraint_rows[:, entering_col_index]

    leaving_row_index = -1
    best_ratio = None

    for row_index in range(constraint_rows.shape[0]):
        coefficient = entering_column[row_index]
        if coefficient > EPS:
            ratio = rhs_column[row_index] / coefficient
            if (best_ratio is None or
                    ratio < best_ratio - 1e-12 or
                    (abs(ratio - best_ratio) <= 1e-12 and
   row_index < leaving_row_index)):
                best_ratio = ratio
                leaving_row_index = row_index

    is_unbounded = leaving_row_index == -1
    return {
        "is_unbounded": is_unbounded,
        "leaving_row_index": leaving_row_index
    }


def simplex_maximize(tableau, basis_variable_indices,
   max_iterations=1000):
    status = "iter_limit"
    iteration = 0
```

```
    while iteration < max_iterations:
        entering_info = choose_entering_variable_max(tableau)

        if entering_info["is_optimal"]:
            status = "optimal"
            break

        entering_col_index = entering_info["entering_col_index"]
        leaving_info = choose_leaving_variable(tableau,
   entering_col_index)

        if leaving_info["is_unbounded"]:
            status = "unbounded"
            break

        leaving_row_index = leaving_info["leaving_row_index"]
        pivot_tableau(tableau, leaving_row_index,
   entering_col_index)
        basis_variable_indices[leaving_row_index] =
   entering_col_index

        iteration += 1

    return {
        "status": status,
        "tableau": tableau,
        "basis": basis_variable_indices
    }


def build_phase1_tableau_with_artificial(A_matrix, rhs_vector):
    constraint_count, variable_count = A_matrix.shape
    artificial_identity = np.eye(constraint_count)

    augmented_matrix = np.hstack([A_matrix.astype(float),
   artificial_identity])
    total_variable_count = variable_count + constraint_count

    tableau = np.zeros((constraint_count + 1,
   total_variable_count + 1), dtype=float)
```

```
    tableau[:constraint_count, :total_variable_count] =
   augmented_matrix
    tableau[:constraint_count, -1] = rhs_vector.astype(float)

    phase1_cost = np.zeros(total_variable_count, dtype=float)
    phase1_cost[variable_count:] = -1.0

    basis_variable_indices = [variable_count + i for i in
   range(constraint_count)]

    basis_costs = phase1_cost[basis_variable_indices]
    tableau[-1, :total_variable_count] = (
        basis_costs @ tableau[:constraint_count,
   :total_variable_count]
    ) - phase1_cost
    tableau[-1, -1] = basis_costs @ tableau[:constraint_count,
   -1]

    return {
        "tableau": tableau,
        "basis": basis_variable_indices,
        "original_variable_count": variable_count
    }


def try_remove_artificials_from_basis(tableau,
   basis_variable_indices, original_variable_count):
    constraint_count = tableau.shape[0] - 1

    for row_index in range(constraint_count):
        basis_index = basis_variable_indices[row_index]
        if basis_index >= original_variable_count:
            pivoted = False
            for col_index in range(original_variable_count):
                if abs(tableau[row_index, col_index]) > EPS:
                    pivot_tableau(tableau, row_index, col_index)
                    basis_variable_indices[row_index] = col_index
                    pivoted = True
                    break
            if not pivoted:
                pass
```

```
def build_phase2_tableau(phase1_tableau, phase1_basis,
   objective_cost, original_variable_count):
    constraint_count = phase1_tableau.shape[0] - 1

    tableau = np.zeros((constraint_count + 1,
   original_variable_count + 1), dtype=float)
    tableau[:constraint_count, :original_variable_count] = \
        phase1_tableau[:constraint_count,
   :original_variable_count]
    tableau[:constraint_count, -1] =
   phase1_tableau[:constraint_count, -1]

    basis_variable_indices = list(phase1_basis)

    basis_ok = True
    for row_index in range(constraint_count):
        if basis_variable_indices[row_index] >=
   original_variable_count:
            basis_ok = False

    if basis_ok:
        basis_costs = objective_cost[basis_variable_indices]
        tableau[-1, :original_variable_count] = (
            basis_costs @ tableau[:constraint_count,
   :original_variable_count]
        ) - objective_cost
        tableau[-1, -1] = basis_costs @
   tableau[:constraint_count, -1]

    return {
        "tableau": tableau,
        "basis": basis_variable_indices,
        "basis_ok": basis_ok
    }


def extract_solution(tableau, basis_variable_indices,
   variable_count):
    solution = np.zeros(variable_count, dtype=float)
```

```
    constraint_count = tableau.shape[0] - 1

    for row_index in range(constraint_count):
        basis_var = basis_variable_indices[row_index]
        if basis_var < variable_count:
            solution[basis_var] = tableau[row_index, -1]

    objective_value = float(tableau[-1, -1])
    return {
        "x": solution,
        "z": objective_value
    }


def solve_two_phase(A_matrix, rhs_vector, objective_cost):
    status = "unknown"
    solution_vector = None
    objective_value = None

    phase1_data = build_phase1_tableau_with_artificial(A_matrix,
   rhs_vector)
    phase1_result = simplex_maximize(phase1_data["tableau"],
   phase1_data["basis"])

    phase1_status = phase1_result["status"]
    phase1_tableau = phase1_result["tableau"]
    phase1_basis = phase1_result["basis"]
    original_variable_count =
   phase1_data["original_variable_count"]

    if phase1_status != "optimal":
        status = phase1_status
    else:
        phase1_objective = float(phase1_tableau[-1, -1])

        feasible = abs(phase1_objective) <= 1e-8
        if not feasible:
            status = "infeasible"
        else:
            try_remove_artificials_from_basis(
                phase1_tableau,
```

```
                phase1_basis,
                original_variable_count
            )

            phase2_data = build_phase2_tableau(
                phase1_tableau,
                phase1_basis,
                objective_cost.astype(float),
                original_variable_count
            )

            if not phase2_data["basis_ok"]:
                status = "infeasible"
            else:
                phase2_result = simplex_maximize(
                    phase2_data["tableau"],
                    phase2_data["basis"]
                )

                status = phase2_result["status"]

                if status == "optimal":
                    solution_data = extract_solution(
                        phase2_result["tableau"],
                        phase2_result["basis"],
                        original_variable_count
                    )
                    solution_vector = solution_data["x"]
                    objective_value = solution_data["z"]

    return {
        "status": status,
        "x": solution_vector,
        "z": objective_value
    }


def main():
    A_matrix = np.array([
        [1, 5, -3, -4, 2, 1],
        [2, 9, -5, -7, 4, 2]
```

```
    ], dtype=float)

    rhs_vector = np.array([14, 32], dtype=float)

    objective_cost = np.array([1, -3, 4, 5, -1, 8], dtype=float)

    result = solve_two_phase(A_matrix, rhs_vector,
   objective_cost)

    print("Метод искусственного базиса")
    print("Статус:", result["status"])
    if result["status"] == "optimal":
        print("x:", np.round(result["x"], 6))
        print("z:", round(result["z"], 6))


if __name__ == "__main__":
    main()
```

Результат работы:

```
Метод искусственного базиса
Статус: optimal
x: [ 0.  0.  0.  4.  0. 30.]
z: 260.0
```

Рис.1: Результат работы программы для метода искусственного базиса.

Блок-схемы метода искусственного базиса:

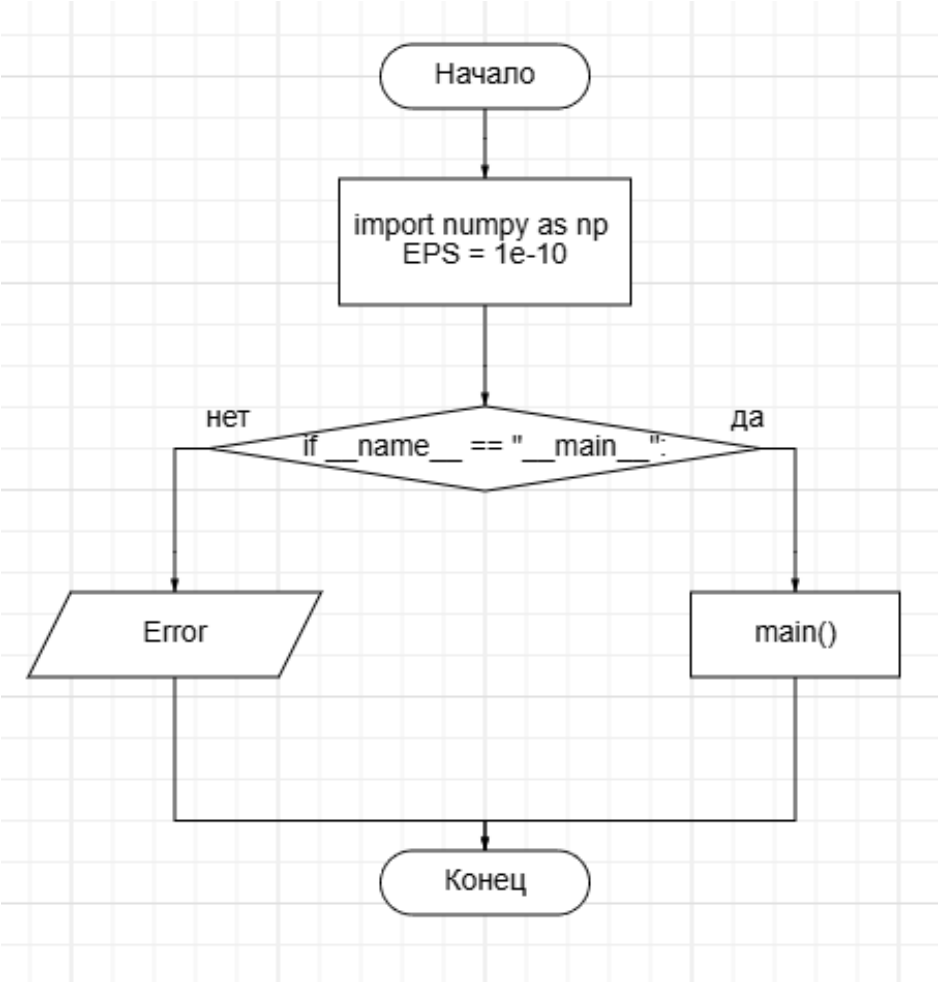

Рис.2: Блок-схема main.

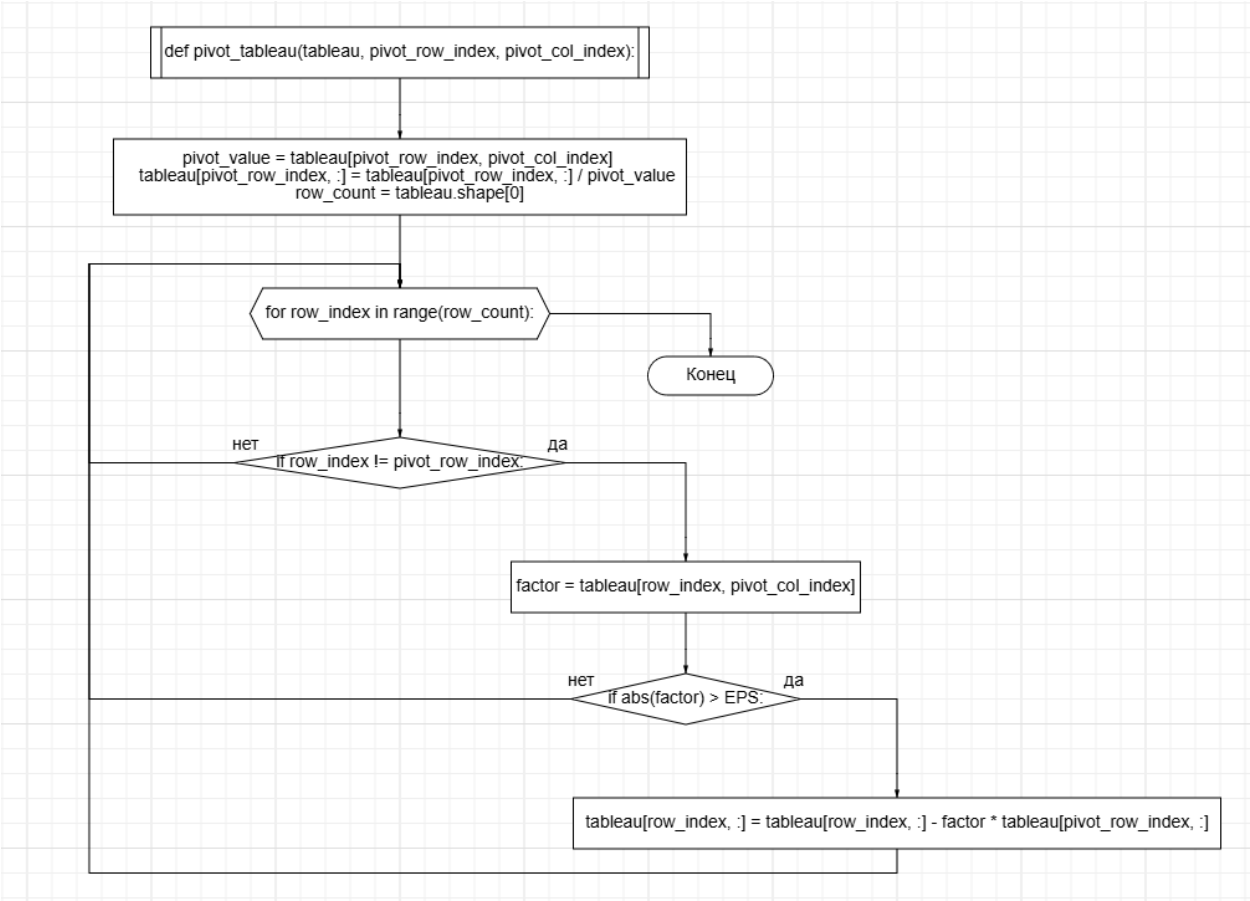

Рис.3: Блок-схема функции pivot_tableau.

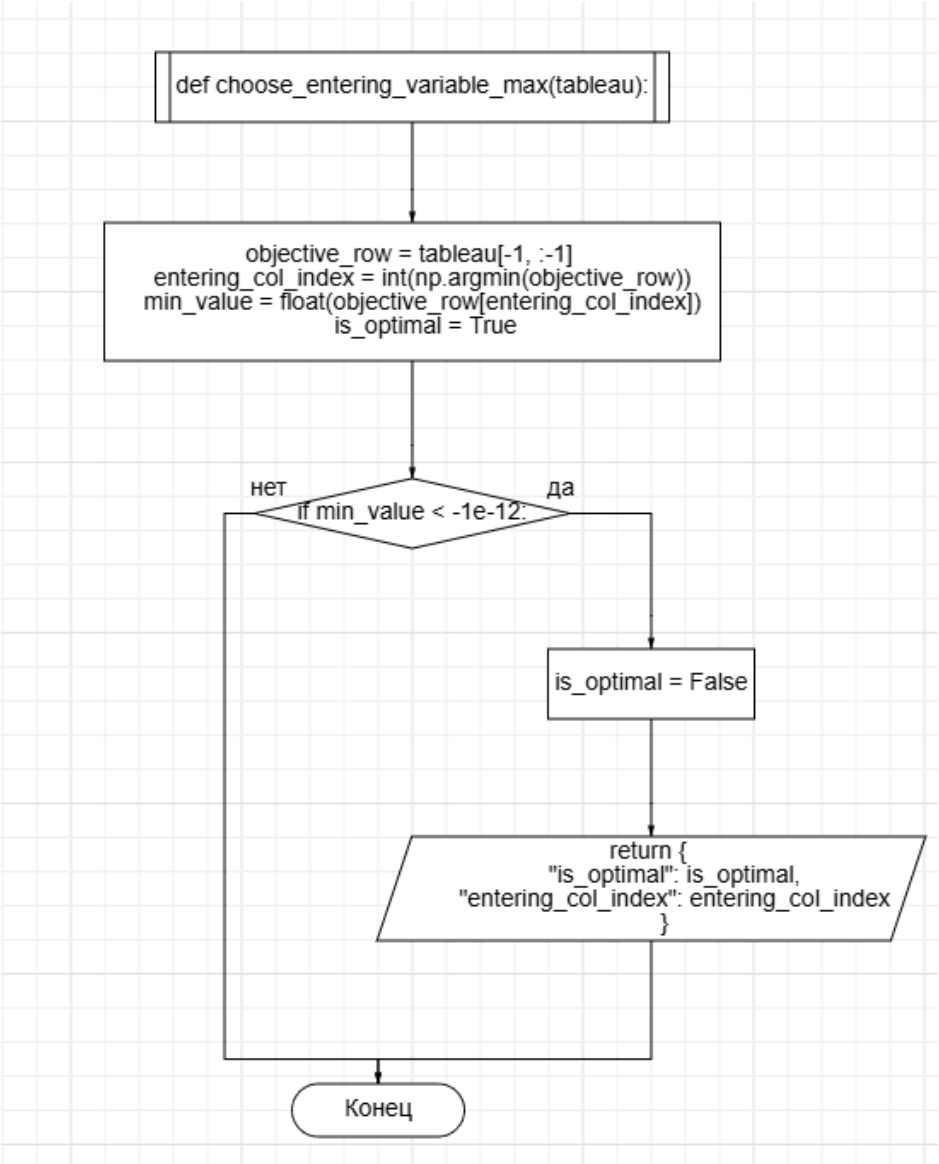

Рис.4: Блок-схема функции choose_entering_variable_max.

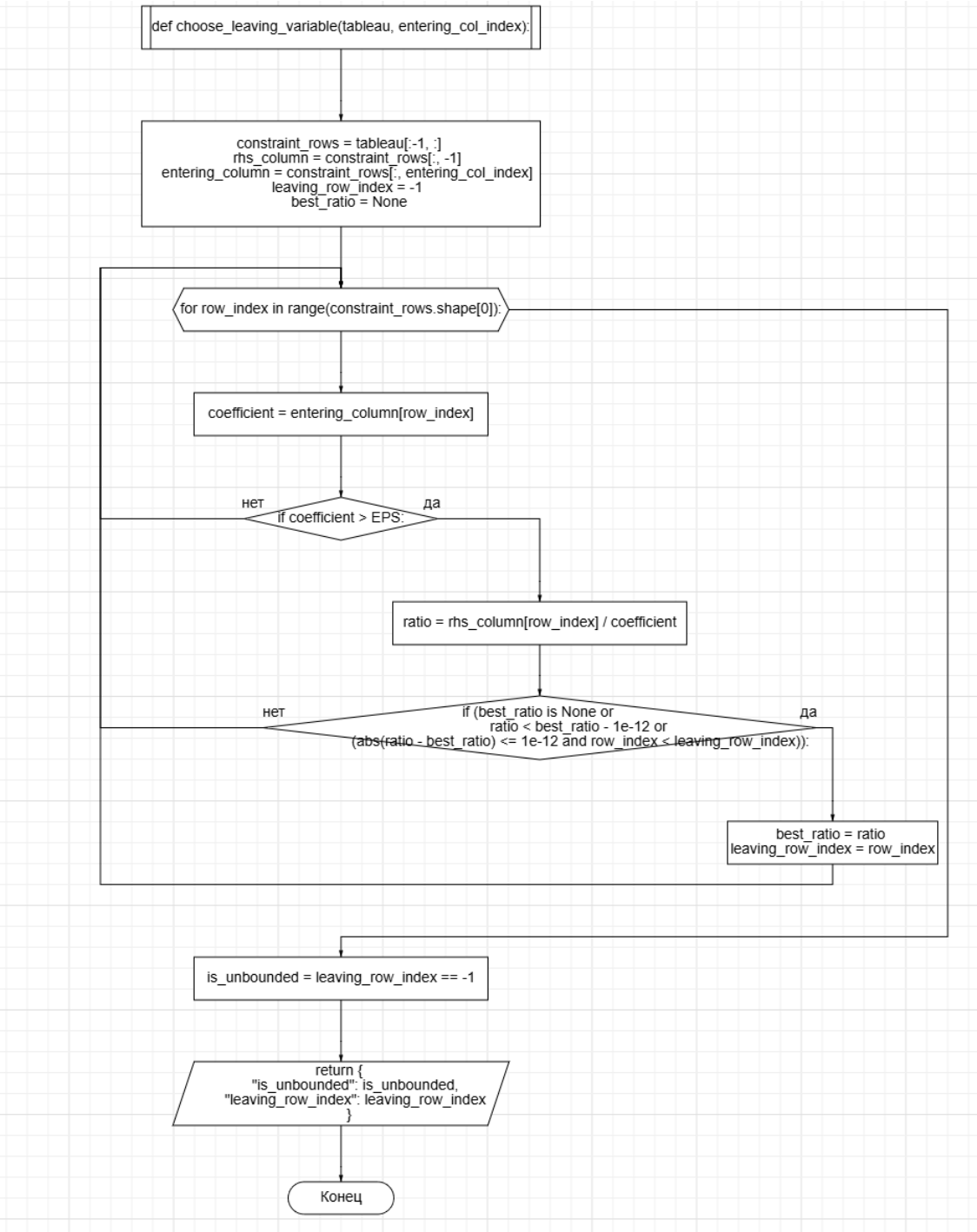

Рис.5: Блок-схема функции choose_leaving_variable.

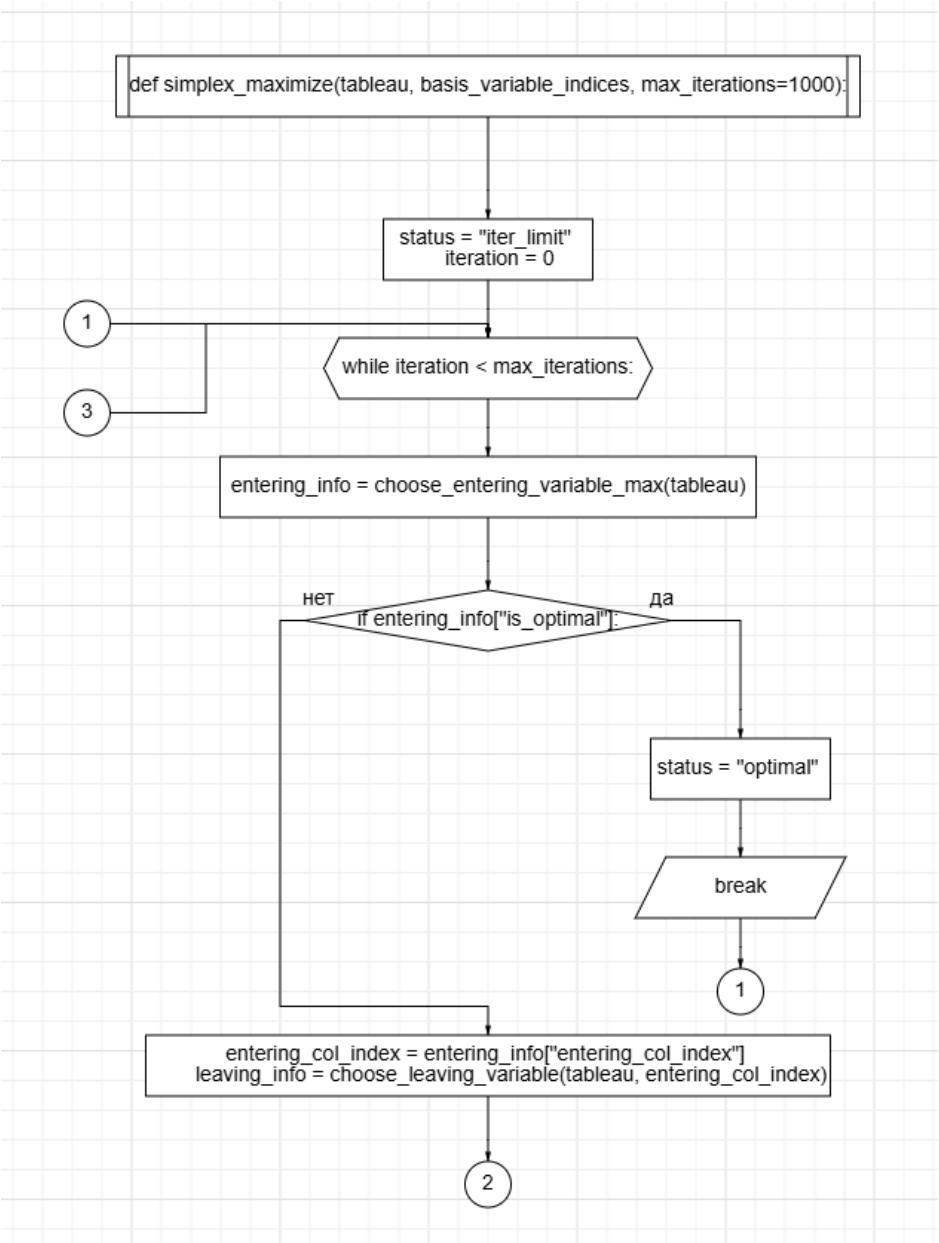

Рис.6: Блок-схема функции simplex_maximize.

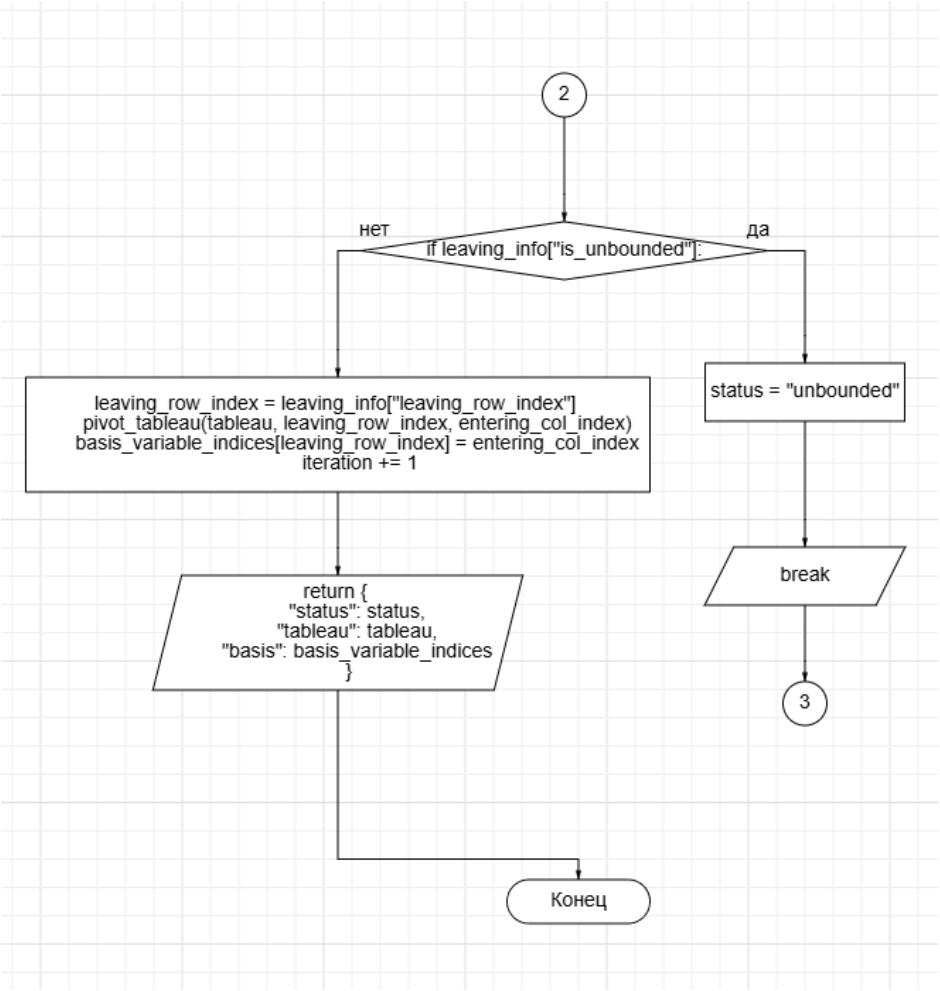

Рис.7: Продолжение блок-схемы функции simplex_maximize.

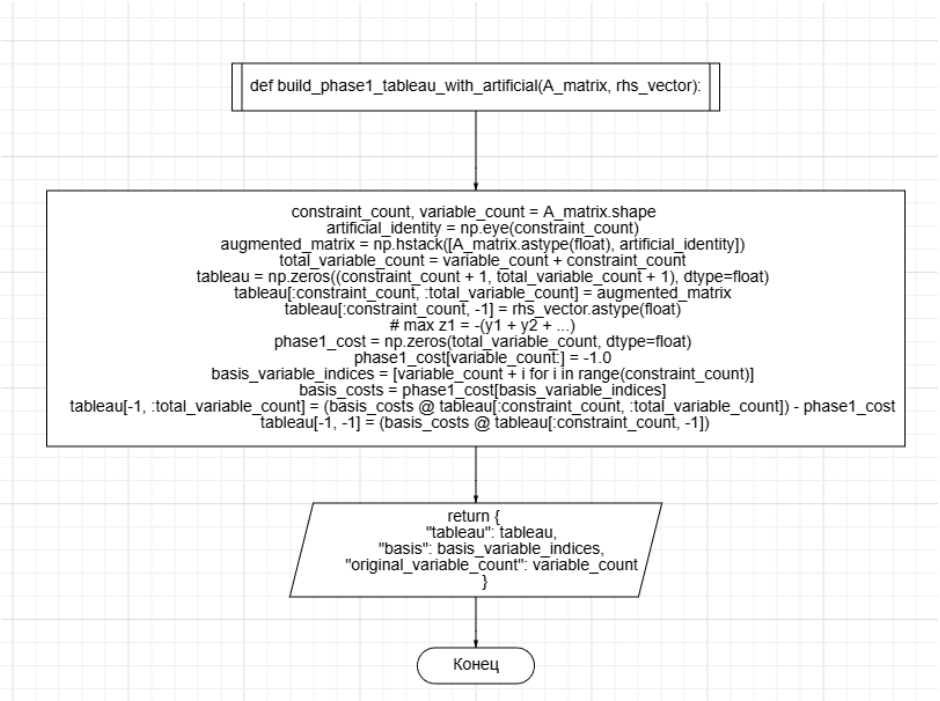

Рис.8: Блок-схема функции build_phase1_tableau_with_artificial.

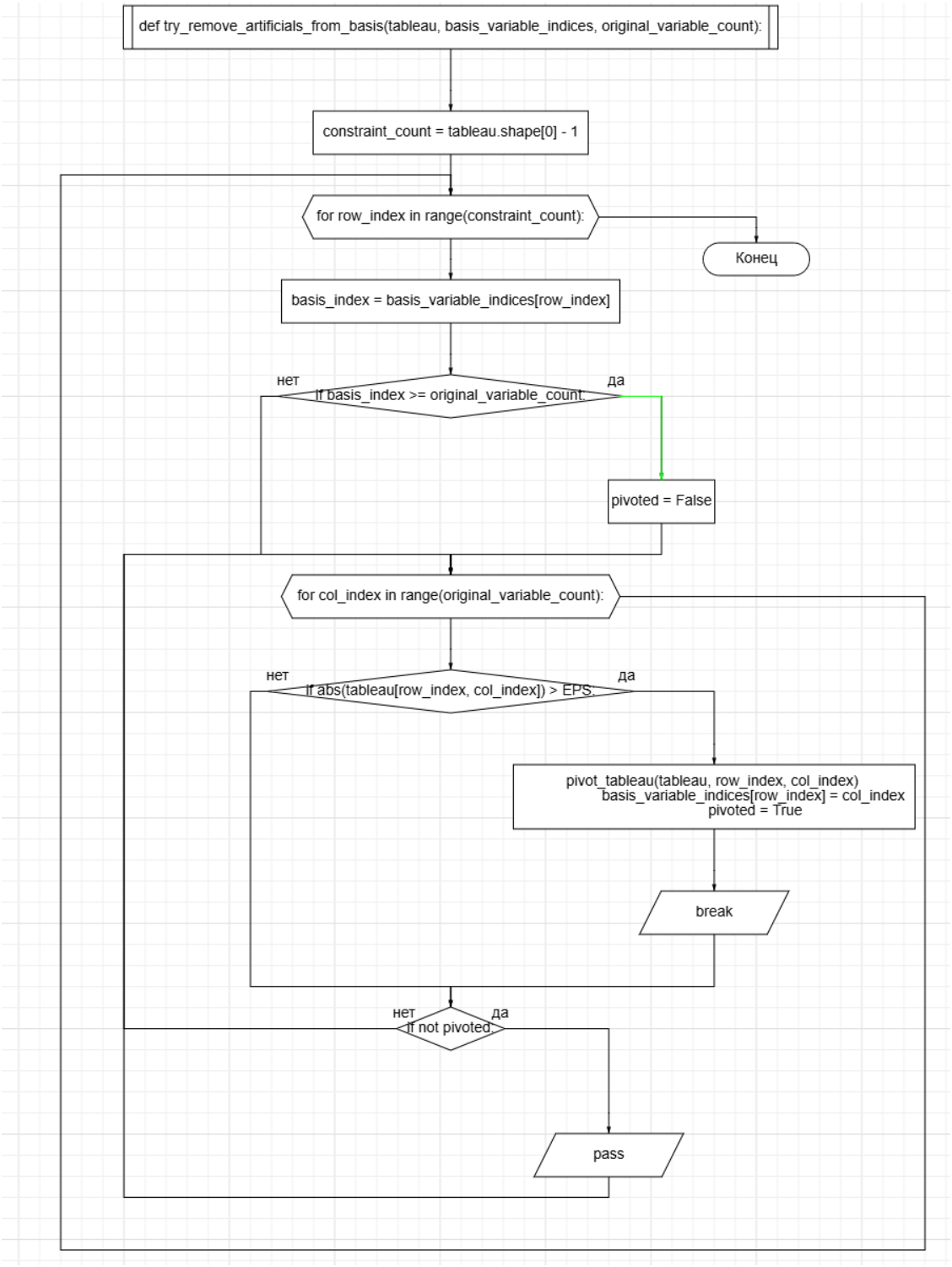

Рис.9: Блок-схема функции try_remove_artificials_from_basis.

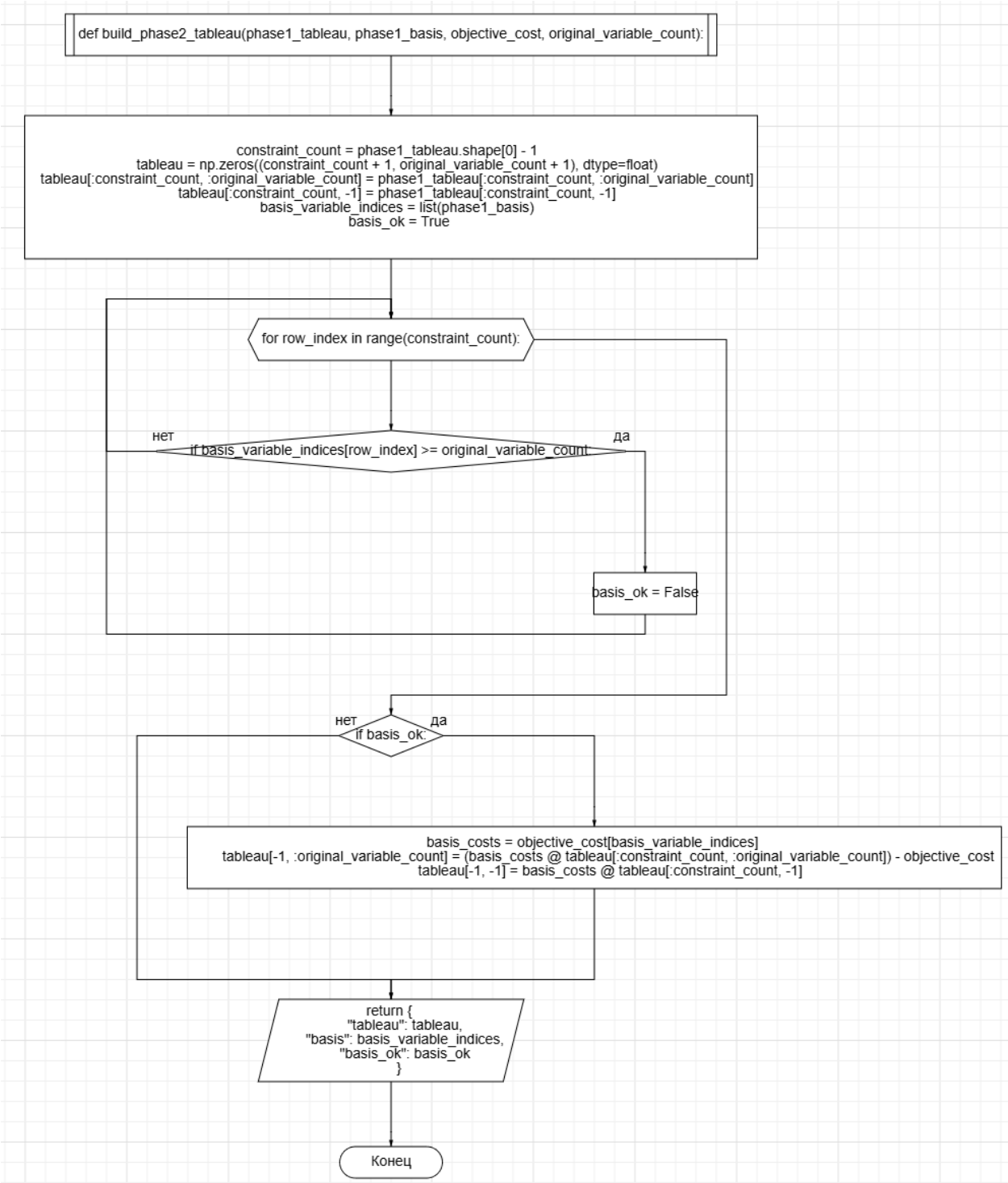

Рис.10: Блок-схема функции build_phase2_tableau.

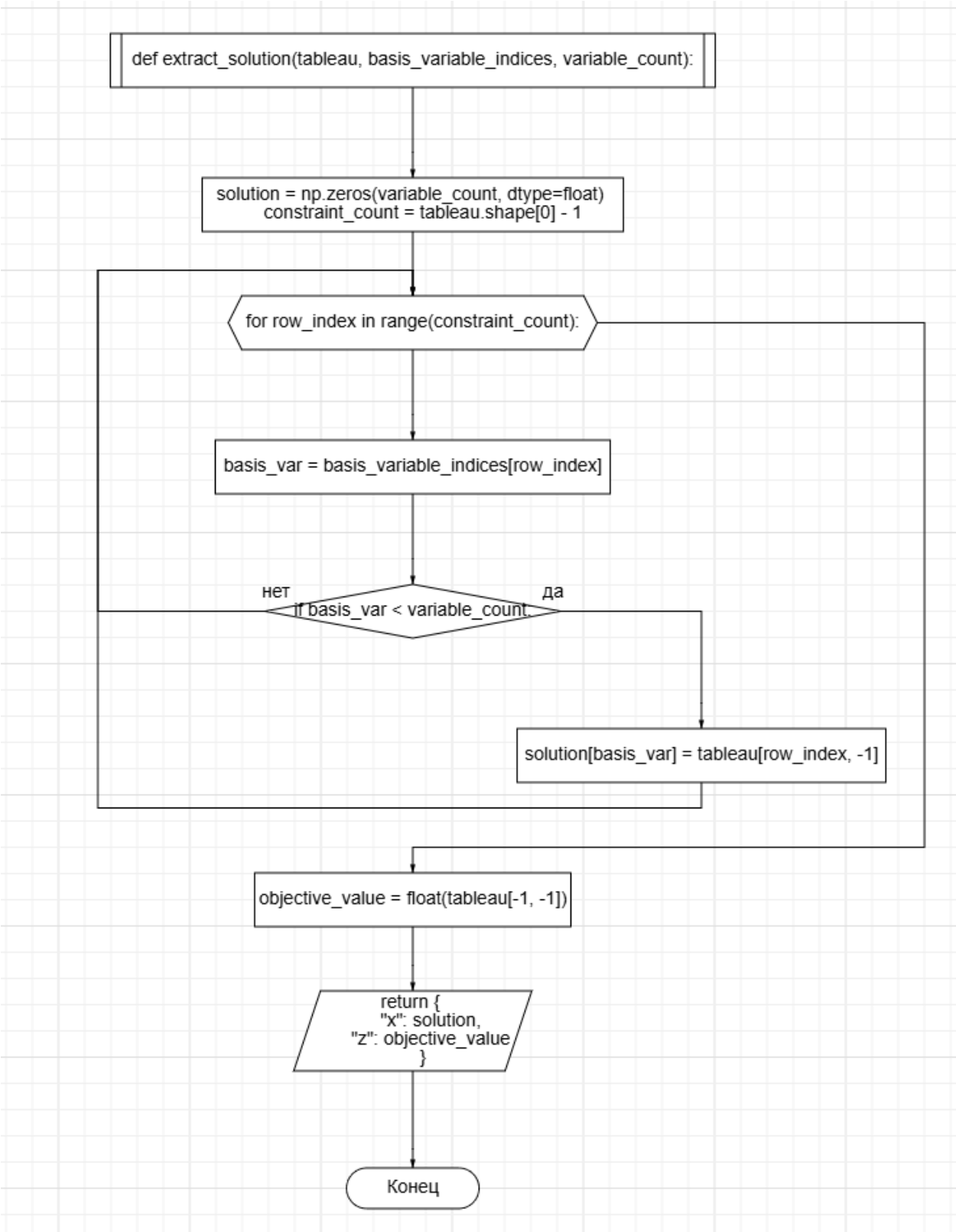

Рис.11: Блок-схема функции extract_solution.

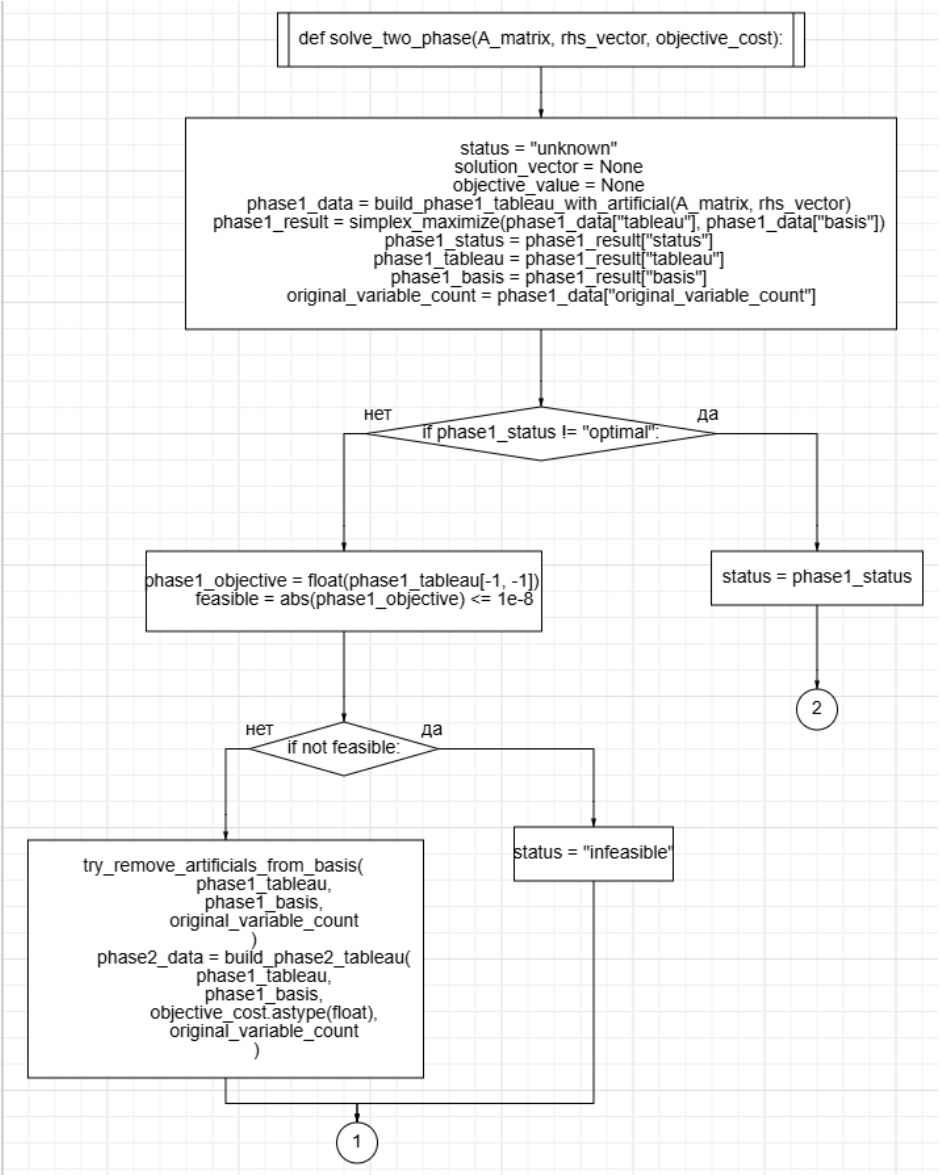

Рис.12: Блок-схема функции solve_two_phase.

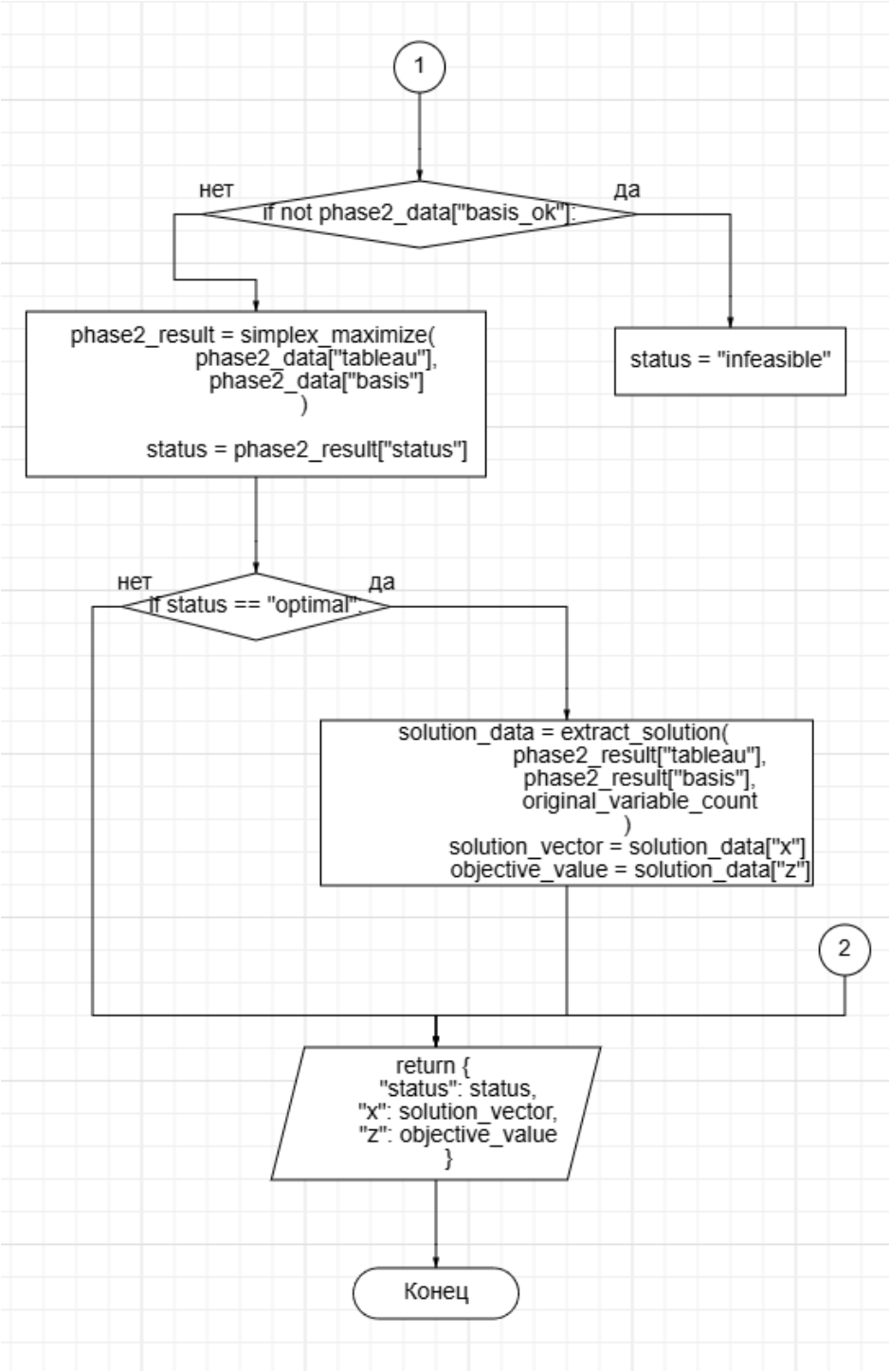

Рис.13: Продолжение блок-схемы функции solve_two_phase.

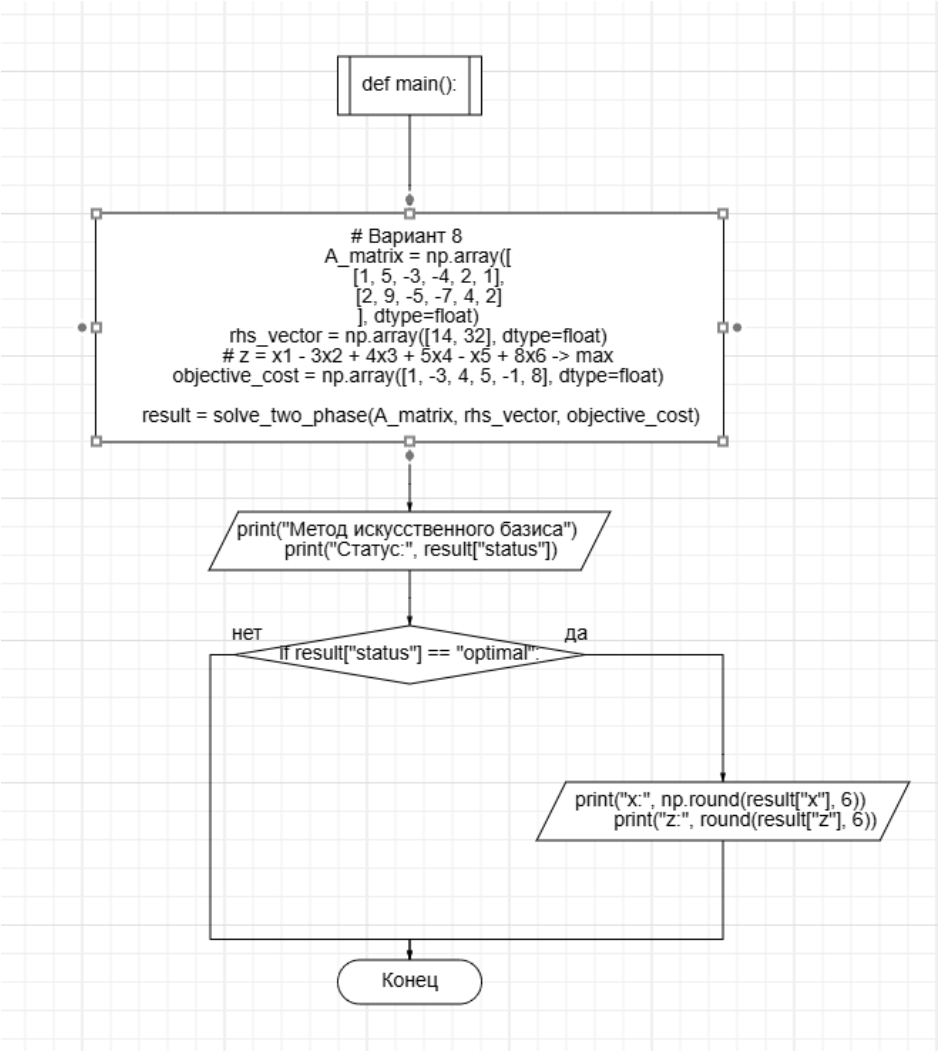

Рис.14: Блок-схема функции main.

Код программы метода больших штрафов:

```
import numpy as np

EPS = 1e-10


def pivot_tableau(tableau, pivot_row_index, pivot_col_index):
    pivot_value = tableau[pivot_row_index, pivot_col_index]
    tableau[pivot_row_index, :] = tableau[pivot_row_index, :] /
   pivot_value

    row_count = tableau.shape[0]
    for row_index in range(row_count):
        if row_index != pivot_row_index:
            factor = tableau[row_index, pivot_col_index]
            if abs(factor) > EPS:
```

```
                tableau[row_index, :] = tableau[row_index, :] -
factor * tableau[pivot_row_index, :]


def choose_entering_variable_max(tableau):
    objective_row = tableau[-1, :-1]
    entering_col_index = int(np.argmin(objective_row))
    min_value = float(objective_row[entering_col_index])

    is_optimal = True
    if min_value < -1e-12:
        is_optimal = False

    return {
        "is_optimal": is_optimal,
        "entering_col_index": entering_col_index
    }


def choose_leaving_variable(tableau, entering_col_index):
    constraint_rows = tableau[:-1, :]
    rhs_column = constraint_rows[:, -1]
    entering_column = constraint_rows[:, entering_col_index]

    leaving_row_index = -1
    best_ratio = None

    for row_index in range(constraint_rows.shape[0]):
        coefficient = entering_column[row_index]
        if coefficient > EPS:
            ratio = rhs_column[row_index] / coefficient
            if (best_ratio is None or
                    ratio < best_ratio - 1e-12 or
                    (abs(ratio - best_ratio) <= 1e-12 and
row_index < leaving_row_index)):
                best_ratio = ratio
                leaving_row_index = row_index

    is_unbounded = leaving_row_index == -1
    return {
        "is_unbounded": is_unbounded,
```

```
        "leaving_row_index": leaving_row_index
    }


def simplex_maximize(tableau, basis_variable_indices,
   max_iterations=1000):
    status = "iter_limit"
    iteration = 0

    while iteration < max_iterations:
        entering_info = choose_entering_variable_max(tableau)

        if entering_info["is_optimal"]:
            status = "optimal"
            break

        entering_col_index = entering_info["entering_col_index"]
        leaving_info = choose_leaving_variable(tableau,
   entering_col_index)

        if leaving_info["is_unbounded"]:
            status = "unbounded"
            break

        leaving_row_index = leaving_info["leaving_row_index"]
        pivot_tableau(tableau, leaving_row_index,
   entering_col_index)
        basis_variable_indices[leaving_row_index] =
   entering_col_index

        iteration += 1

    return {
        "status": status,
        "tableau": tableau,
        "basis": basis_variable_indices
    }


def extract_solution(tableau, basis_variable_indices,
   variable_count):
```

```
    solution = np.zeros(variable_count, dtype=float)
    constraint_count = tableau.shape[0] - 1

    for row_index in range(constraint_count):
        basis_var = basis_variable_indices[row_index]
        if basis_var < variable_count:
            solution[basis_var] = tableau[row_index, -1]

    return {
        "x": solution,
        "z": float(tableau[-1, -1])
    }


def solve_big_m(A_matrix, rhs_vector, objective_cost,
   big_m_value=1e6):
    status = "unknown"
    solution_vector = None
    objective_value = None

    constraint_count, variable_count = A_matrix.shape
    artificial_identity = np.eye(constraint_count)

    augmented_matrix = np.hstack([A_matrix.astype(float),
   artificial_identity])
    augmented_cost = np.concatenate([
        objective_cost.astype(float),
        -big_m_value * np.ones(constraint_count)
    ])

    total_variable_count = variable_count + constraint_count

    tableau = np.zeros((constraint_count + 1,
   total_variable_count + 1), dtype=float)
    tableau[:constraint_count, :total_variable_count] =
   augmented_matrix
    tableau[:constraint_count, -1] = rhs_vector.astype(float)

    basis_variable_indices = [variable_count + i for i in
   range(constraint_count)]
```

```
    basis_costs = augmented_cost[basis_variable_indices]
    tableau[-1, :total_variable_count] = (
        basis_costs @ tableau[:constraint_count,
   :total_variable_count]
    ) - augmented_cost
    tableau[-1, -1] = basis_costs @ tableau[:constraint_count,
   -1]

    simplex_result = simplex_maximize(tableau,
   basis_variable_indices)

    status = simplex_result["status"]
    final_tableau = simplex_result["tableau"]
    final_basis = simplex_result["basis"]

    if status == "optimal":
        extended_solution = np.zeros(total_variable_count,
   dtype=float)
        for row_index in range(constraint_count):
            extended_solution[final_basis[row_index]] =
   final_tableau[row_index, -1]

        artificial_part = extended_solution[variable_count:]
        infeasible = np.max(np.abs(artificial_part)) > 1e-8

        if infeasible:
            status = "infeasible"
        else:
            solution_vector = extended_solution[:variable_count]
            objective_value = float(objective_cost @
   solution_vector)

    return {
        "status": status,
        "x": solution_vector,
        "z": objective_value
    }


def main():
    A_matrix = np.array([
```

```
        [1, 5, -3, -4, 2, 1],
        [2, 9, -5, -7, 4, 2]
    ], dtype=float)

    rhs_vector = np.array([14, 32], dtype=float)

    objective_cost = np.array([1, -3, 4, 5, -1, 8], dtype=float)

    result = solve_big_m(A_matrix, rhs_vector, objective_cost,
   big_m_value=1e6)

    print("Метод больших штрафов")
    print("Статус:", result["status"])
    if result["status"] == "optimal":
        print("x:", np.round(result["x"], 6))
        print("z:", round(result["z"], 6))


if __name__ == "__main__":
    main()
```

Результат работы программы:

```
Метод больших штрафов
Статус: optimal
x: [ 0.  0.  0.  4.  0. 30.]
z: 260.0
```

Рис.15: Результат работы программы для метода больших штрафов.

Блок-схемы метода больших штрафов:

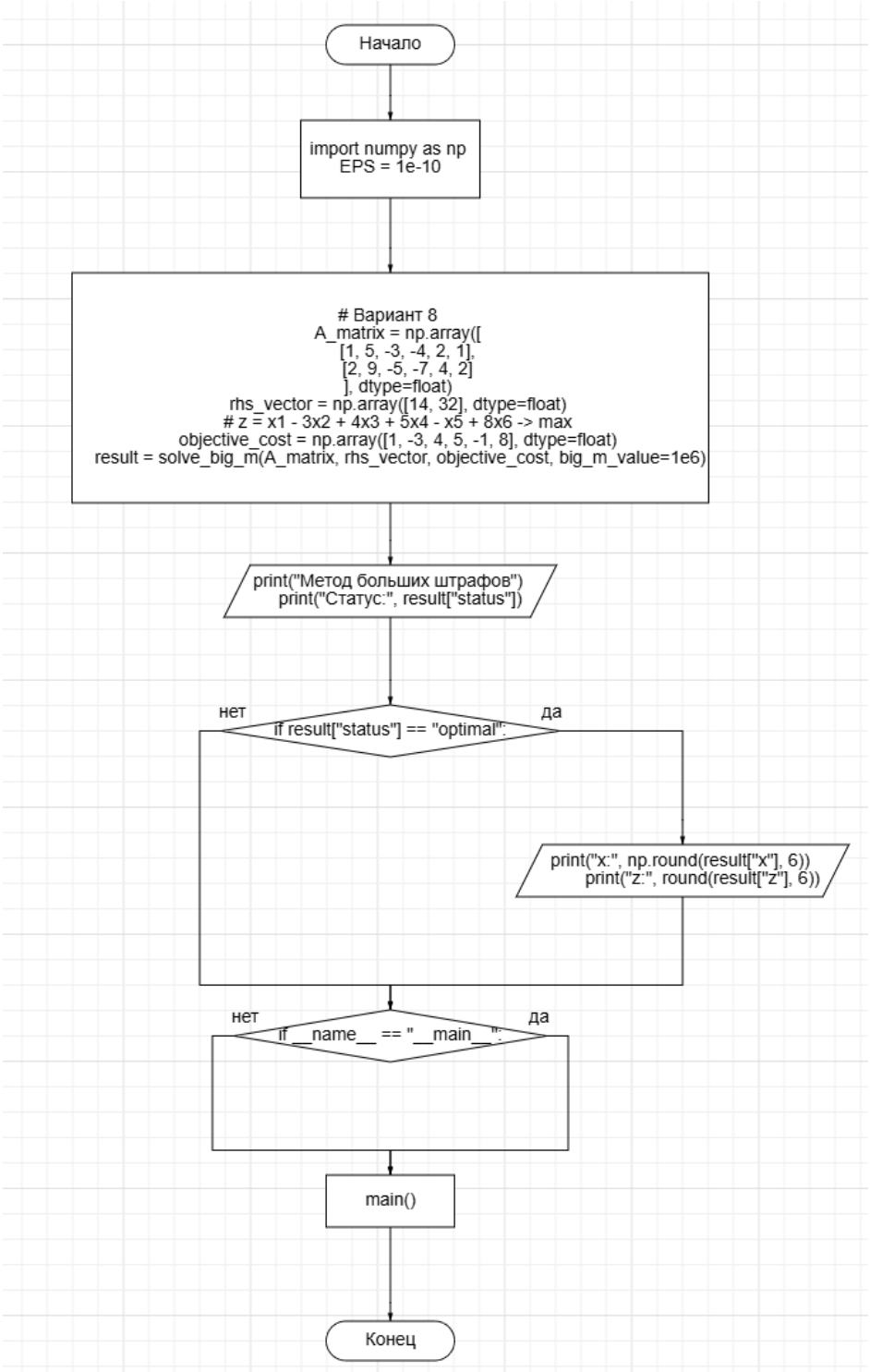

Рис.16: Блок-схема функции main.

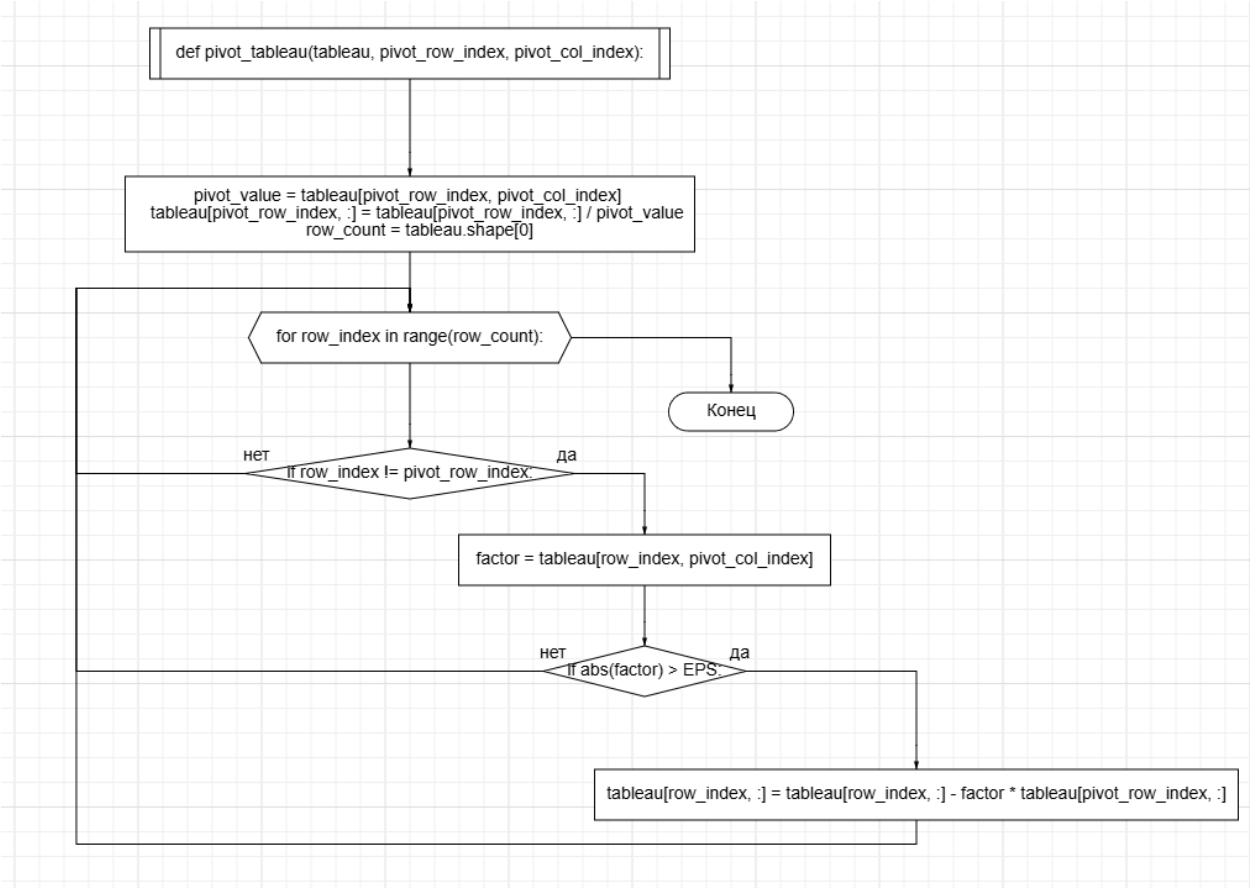

Рис.17: Блок-схема функции pivot_tableau.

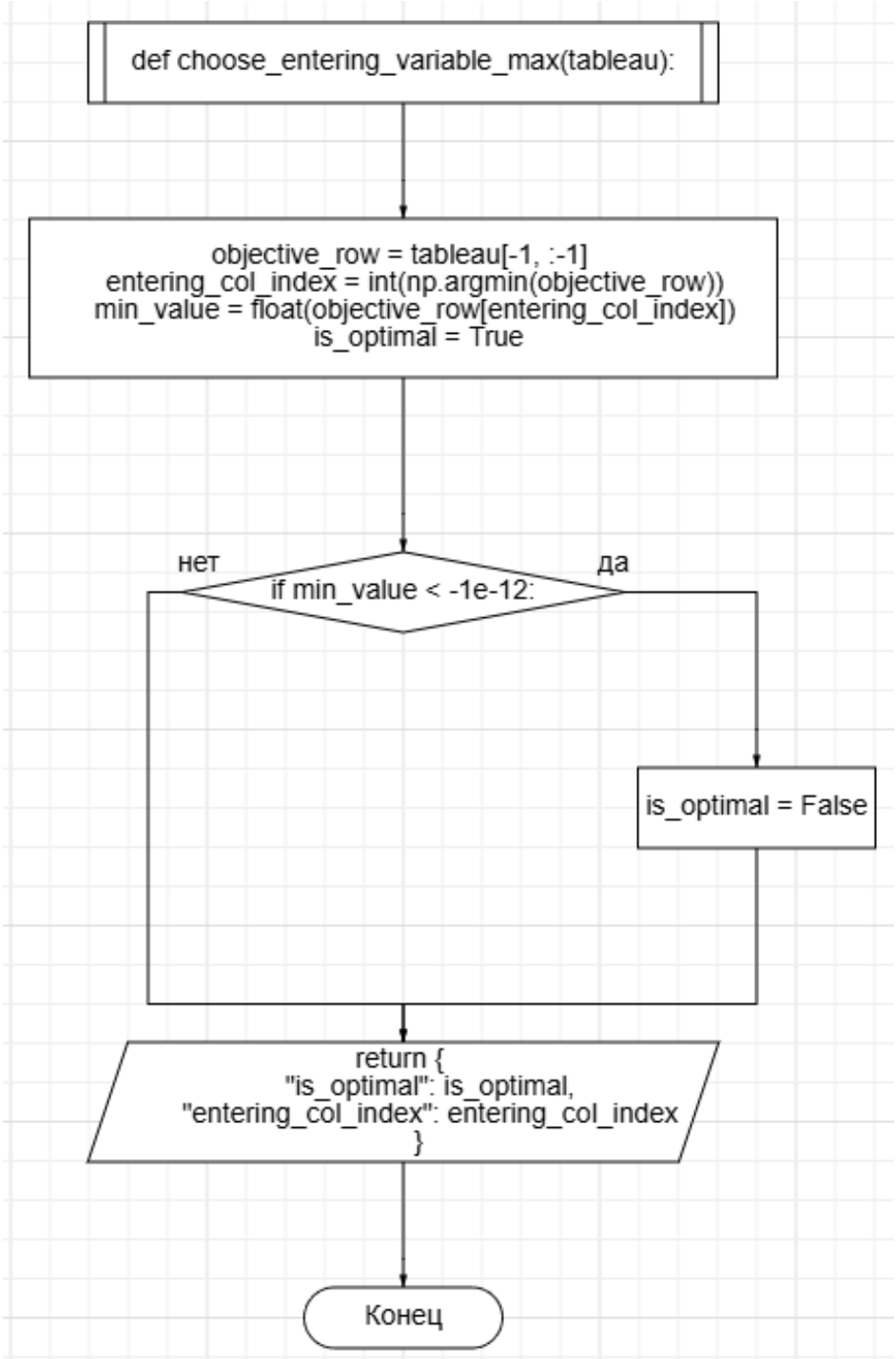

Рис.18: Блок-схема функции choose_entering_variable_max.

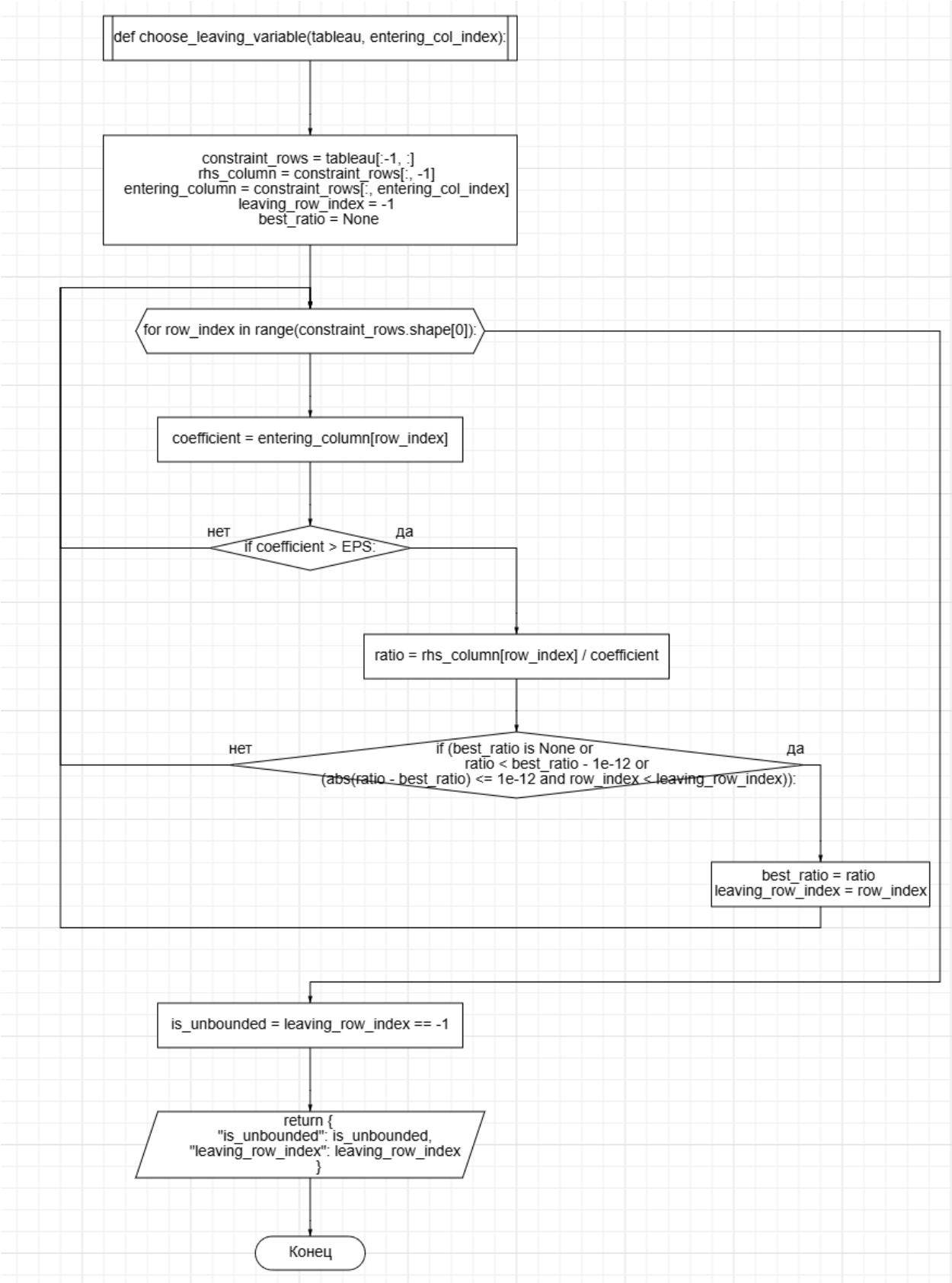

Рис.19: Блок-схема функции choose_leaving_variable.

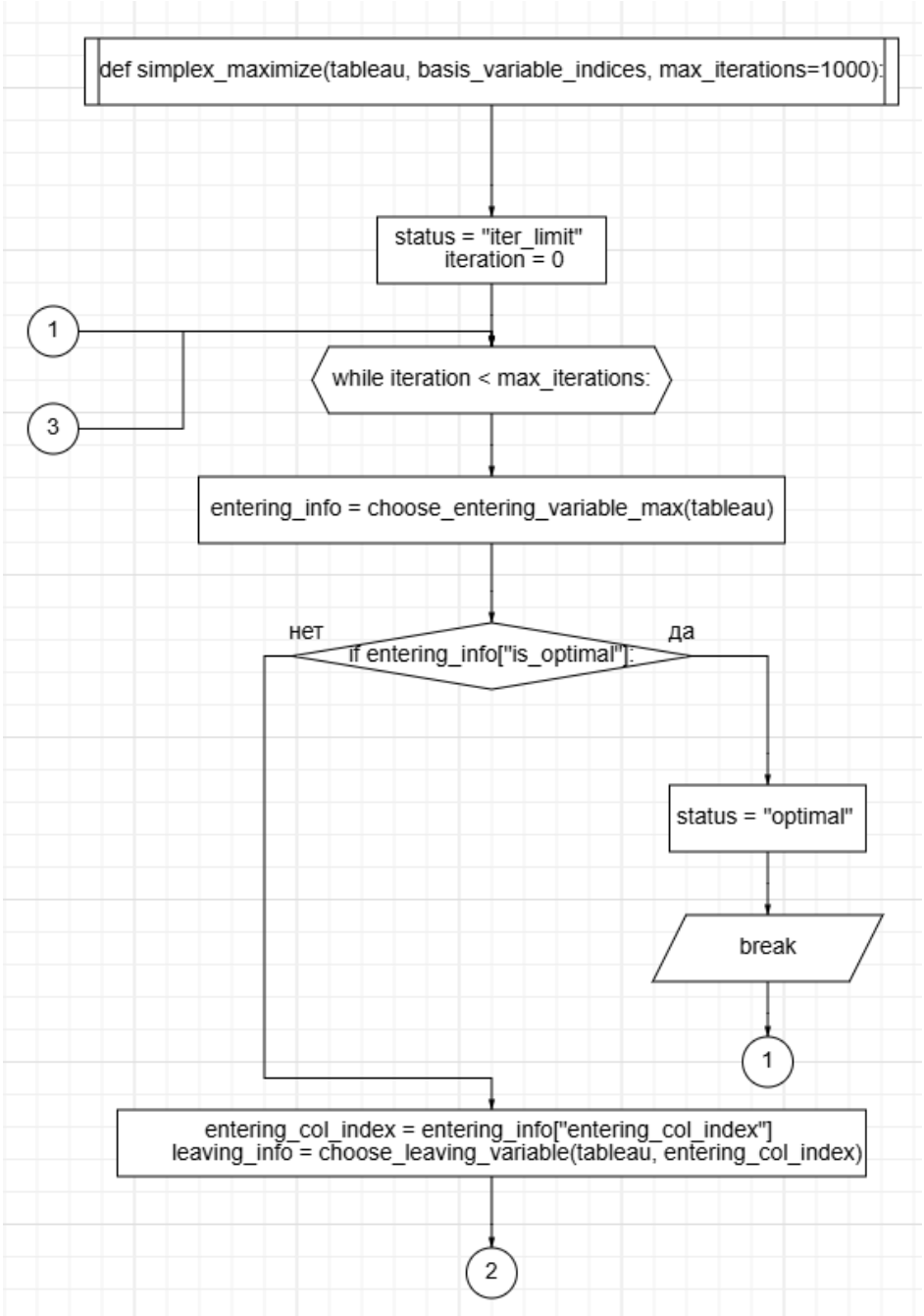

Рис.20: Блок-схема функции simplex_maximize.

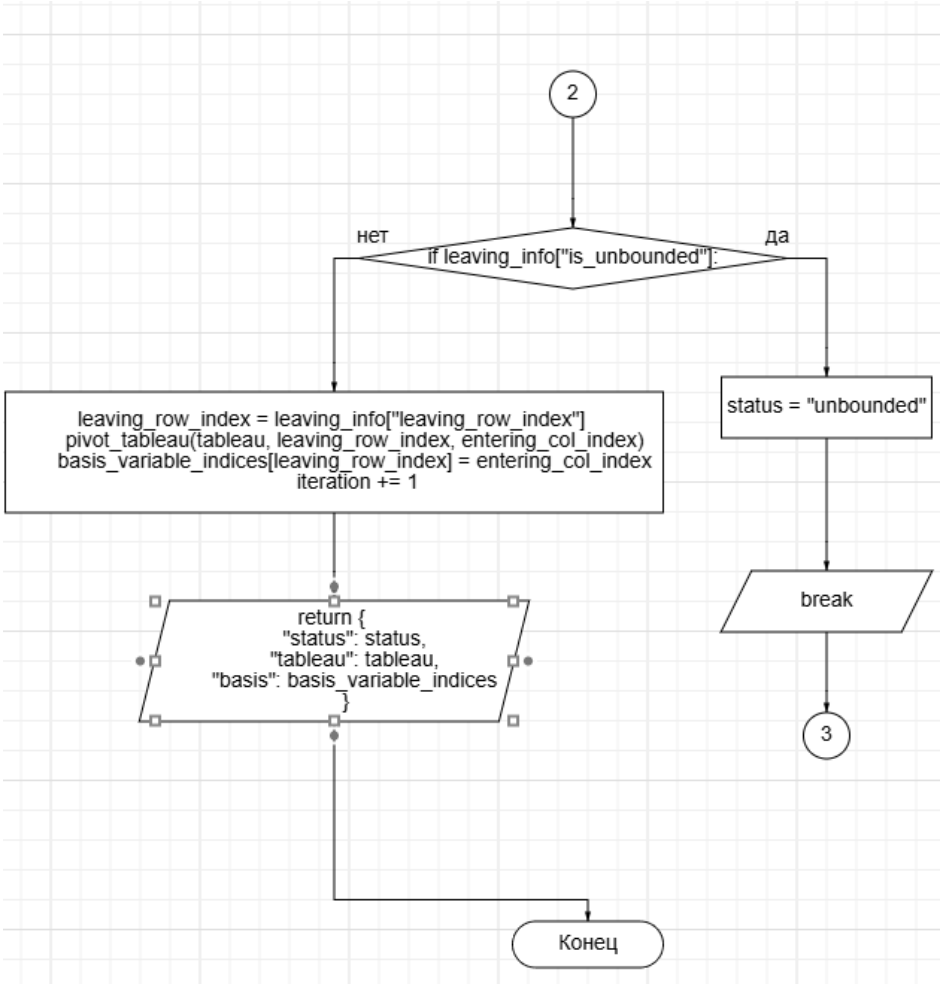

Рис.21: Продолжение блок-схемы функции simplex_maximize.

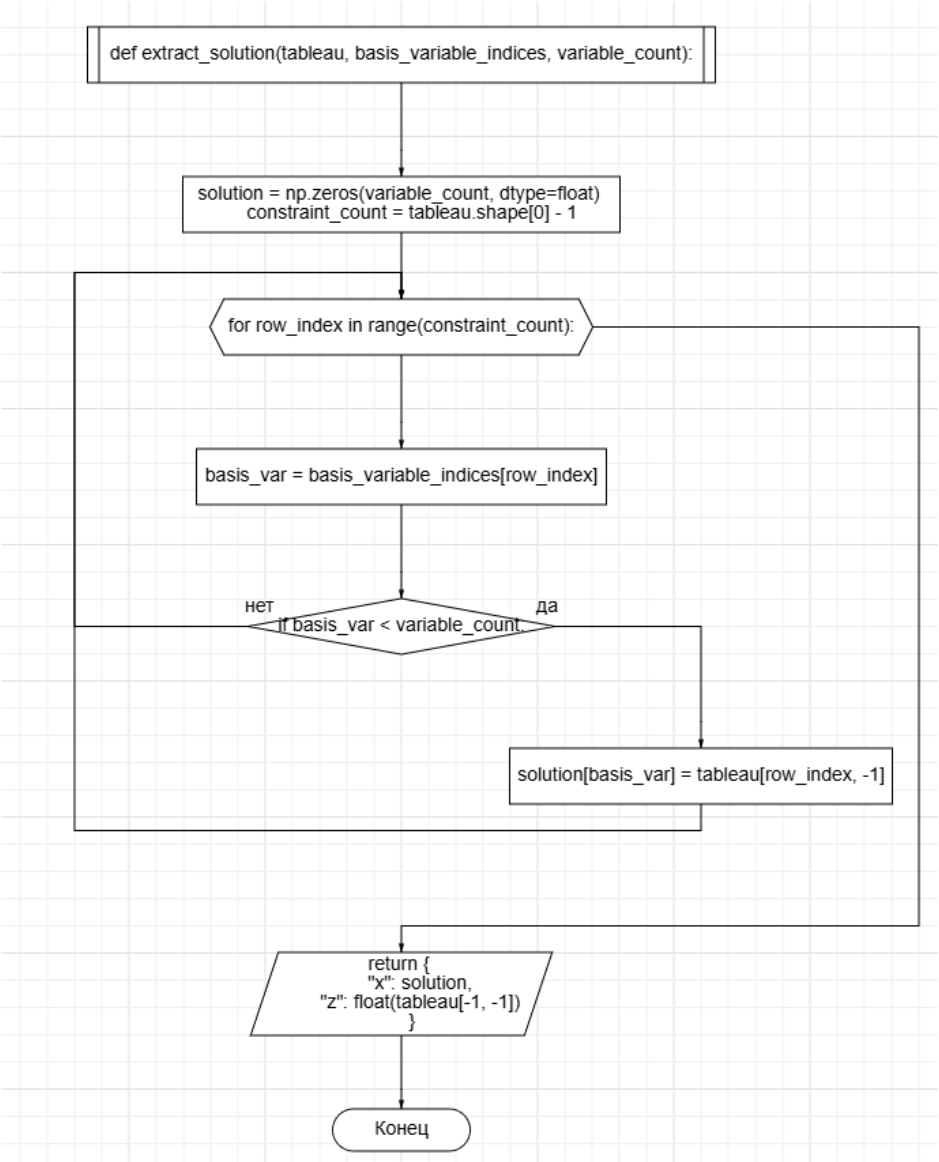

Рис.22: Блок-схема функции extract_solution.

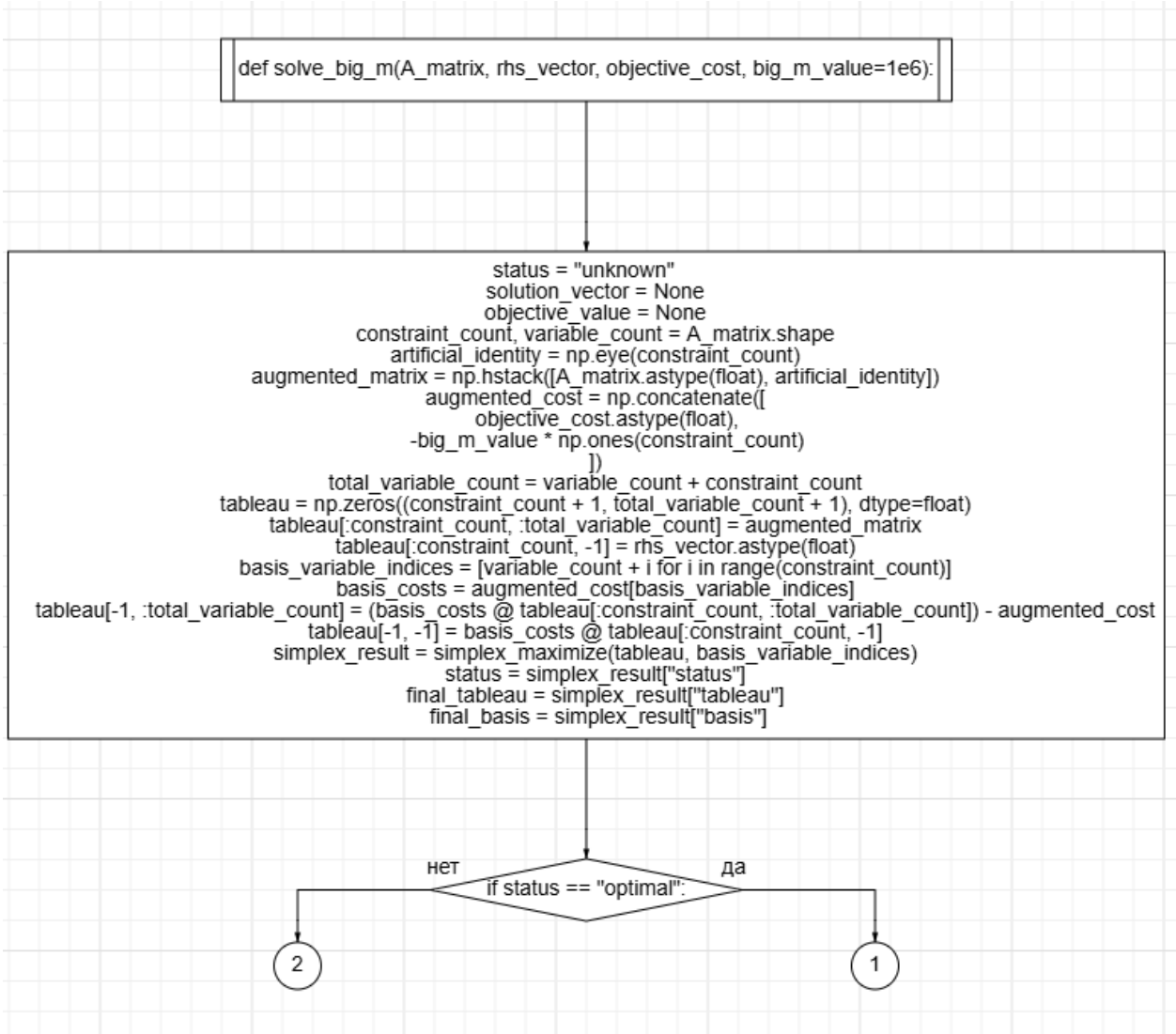

Рис.23: Блок-схема функции solve_big_m.

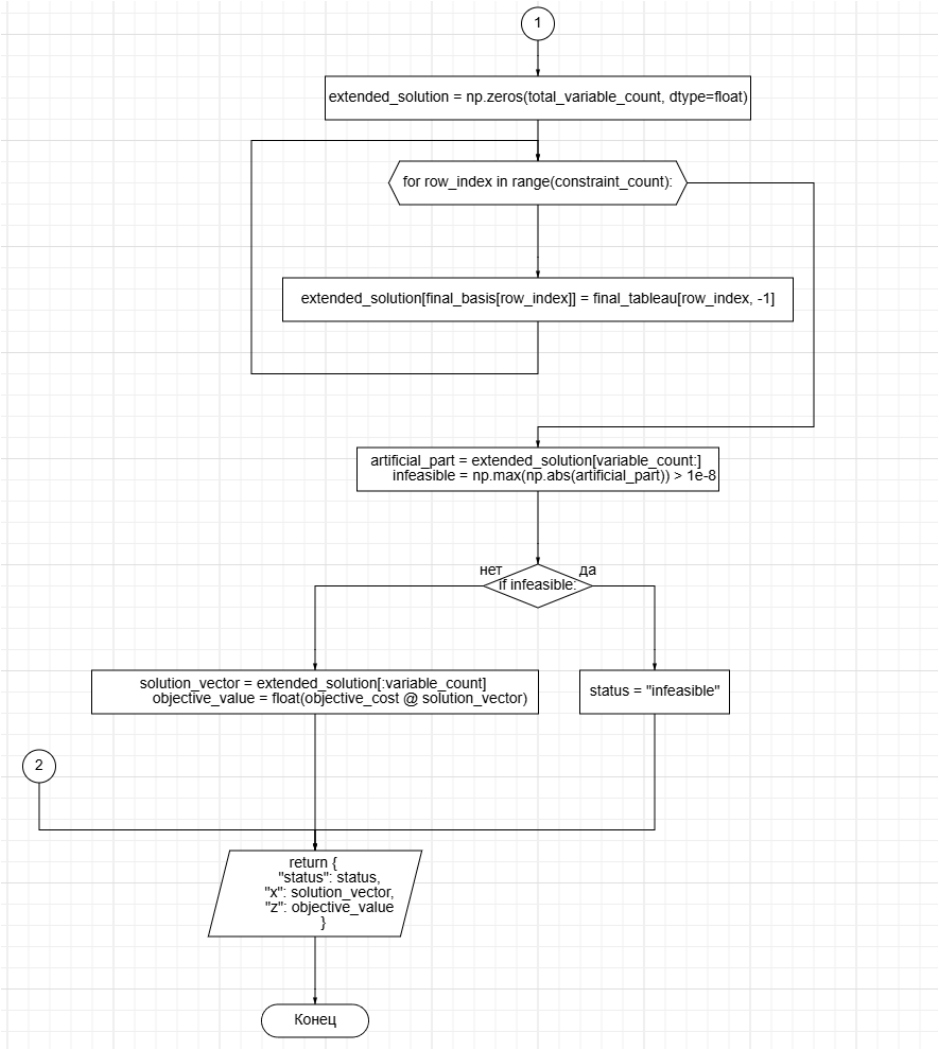

Рис.24: Продолжение блок-схемы функции solve_big_m.

Вывод

В ходе выполнения лабораторной работы были изучены два метода решения задач линейного программирования в канонической форме, не имеющих готового допустимого базисного решения: метод искусственного базиса и метод больших штрафов. Были подробно разобраны принципы построения начальной симплекс-таблицы с введением искусственных переменных, алгоритмы выполнения первой и второй фаз двухфазного метода, а также механизм учёта большого штрафа M в целевой функции. Особое внимание уделялось правилам выбора разрешающего элемента, выполнению pivot-преобразований и проверке условий оптимальности и допустимости решения.

На основе изученных алгоритмов были разработаны и отлажены про-

граммы на языке Python, реализующие оба метода. Программы разбиты на небольшие функциональные блоки, что позволило наглядно представить их в виде блок-схем.

Таким образом, поставленная цель работы достигнута: получены теоретические знания и практические навыки применения методов искусственного базиса и больших штрафов, а также опыт разработки, отладки и тестирования соответствующих алгоритмов.